

1. Đề xuất 2 giải pháp xử lý đường dẫn

- **Giải pháp 1: Đổi thủ công toàn bộ dấu \ thành / trong code**
 - Tiến hành rà soát toàn bộ mã nguồn, tìm kiếm các chuỗi đường dẫn tĩnh (hard-code) sử dụng dấu gạch chéo ngược (\) và thay thế thủ công hoặc dùng tính năng Find & Replace thành gạch chéo xuôi (/) vì Linux và các hệ điều hành hiện đại đều hỗ trợ /.
- **Giải pháp 2: Sử dụng thư viện/hàm chuẩn của ngôn ngữ lập trình để xử lý đường dẫn động**
 - Loại bỏ hoàn toàn các chuỗi đường dẫn cứng, thay thế bằng các module/thư viện chuẩn chuyên dụng của ngôn ngữ lập trình (ví dụ: `path` trong Node.js, `pathlib` hoặc `os.path` trong Python, `java.nio.file` trong Java) để hệ thống tự động nhận diện và sinh dấu phân cách phù hợp với hệ điều hành đang chạy.

2. Bảng so sánh ưu - nhược điểm

Giải pháp	Ưu điểm	Nhược điểm
1. Đổi thủ công (\ thành /)	• Đơn giản, dễ thực hiện ngay lập tức. • Không đòi hỏi thay đổi logic code phức tạp hay tìm hiểu sâu về thư viện.	• Rủi ro cao: Dễ bỏ sót các đoạn code sinh đường dẫn động. • Thiếu tính linh hoạt: Nếu sau này đổi sang hệ điều hành khác hoặc chạy môi trường đa nền tảng (Developer dùng Windows, Production dùng Linux), lỗi có thể tái diễn. • Mất công sức bảo trì thủ công khi codebase phình to.

2. Dùng thư viện chuẩn động	• Chuẩn hóa & Tự động: Tự nhận diện hệ điều hành (\ trên Windows, / trên Linux). • Bền vững: Giúp mã nguồn sạch, dễ mở rộng và an toàn cho các lần di chuyển hệ thống sau. • Giảm thiểu tối đa lỗi do con người (human error).	• Cần thời gian refactor (viết lại) một phần mã nguồn hiện tại. • Đội ngũ lập trình viên cần nắm vững cách sử dụng các hàm chuẩn của ngôn ngữ.
------------------------------------	---	---

3. Lựa chọn giải pháp tối ưu

Giải pháp được chọn: Giải pháp 2 (Sử dụng thư viện/hàm chuẩn của ngôn ngữ lập trình).

Lý do lựa chọn:

Với quy mô hàng nghìn đường dẫn hard-code, việc sửa thủ công (Giải pháp 1) chỉ là giải pháp tạm thời, mang tính "chữa cháy" và chứa rủi ro kỹ thuật rất lớn (dễ sót lỗi đường dẫn khi chạy thực tế trên Ubuntu). Sử dụng thư viện chuẩn giúp CineBook giải quyết triệt để vấn đề không chỉ cho lần migration này mà còn đảm bảo tính ổn định, linh hoạt lâu dài cho các môi trường phát triển (Local vs. Production) sau này.

4. Các bước triển khai giải pháp

1. **Xác định công nghệ và module:** Thống nhất thư viện xử lý đường dẫn chuẩn tương ứng với ngôn ngữ lập trình đang dùng cho CineBook (Ví dụ: `path` trong Node.js, `pathlib` trong Python).
2. **Khoanh vùng mã nguồn cần xử lý:** Sử dụng công cụ tìm kiếm trong IDE (như VS Code, IntelliJ) để lọc ra toàn bộ các file chứa chuỗi đường dẫn tuyệt đối hoặc tương đối có dùng dấu \.
3. **Refactor mã nguồn:** Thay thế các chuỗi hard-code bằng các hàm kết hợp đường dẫn động (Ví dụ: thay vì `"C:\\data\\images"` thì dùng `path.join('data', 'images')`).
4. **Kiểm thử (Testing) trên môi trường Staging:** Triển khai bản dựng lên máy chủ thử nghiệm chạy Linux (Ubuntu) để kiểm tra tính toàn vẹn của việc đọc/ghi file, tải ảnh/tài liệu và xử lý các tệp hệ thống.

